A PRACTICAL, JUDGE-READY TALK TRACK

LabelWise

10-minute pitch, product walkthrough, model and API guide, and judge Q&A;

THE ONE-LINE PITCH

LabelWise helps people compare the food-label information they can see with the dietary needs they share - and says when the label does not give enough evidence.

LABEL PHOTO	VOICE NOTE	PERSONAL PROFILE	CAUTIOUS RESULT

IMPORTANT: MODEL DISCLOSURE

The current live code uses Gemini 3.5 Flash-Lite for label-image analysis and Whisper tiny for voice transcription. It does not currently run Gemma 4. Do not tell judges that Gemma 4 powers this demo unless you actually integrate and verify it before presenting. A ready-to-say honest answer is on page 8.

HOW TO USE THIS PDF

Pages 2-6 are the timed talk track. Pages 7-8 prepare you for judge questions and last-minute checks. The timed talk assumes a short live demo; Q&A; is separate. If the event includes Q&A; in your 10 minutes, use the compressed timing note on page 6.

Prepared from the current LabelWise code and setup. Do not claim accuracy or latency numbers that you have not measured.

Your 10-minute pitch

Speak naturally; the quoted text is a script, not a requirement to memorize every word.

0:00-1:00

Open with the real problem

Imagine standing in a grocery aisle with a peanut allergy or diabetes. The important information is printed in small text across ingredients, allergen warnings, and nutrition facts. It is easy to focus on one number, like sugar, and miss another, like total carbohydrates. It is also easy to mistake an unseen warning for a warning that is not there. That is the problem LabelWise is designed to address.

SHOW / DO: Hold up a package or show a label photo. Pause on the small print.

1:00-2:00

Introduce the solution and USP

LabelWise lets a person upload a food-label photo, tell the app what matters in a voice note, and save allergies or dietary considerations to a personal profile. It returns a plain-language result: Safe, Caution, or Avoid, plus detected ingredients and reasons. The key idea is not just reading a label. It is comparing the visible evidence with the needs a user has shared, while making uncertainty visible. If an important part of the package is missing, the app should say it cannot confirm safety.

SHOW / DO: Show the landing page, then the account profile fields and the three verdict labels.

USP IN ONE SENTENCE

A personalized label reader that combines a package photo, a spoken question, and a saved dietary profile, then defaults to caution when key evidence is missing.

Transition: 'Let me show you how it behaves on a label where the important details are easy to misread.'

Live demo: make the safety behavior visible

Target time: 2:00-4:15. Use a prepared sample and avoid relying on a fresh account setup during the pitch.

Step 1

Set the user's needs

I will use a profile with peanut allergy and diabetes. In a real scan, the user can save these needs in their account or state them in a voice note.

SHOW / DO: Show the profile fields already filled in. Do not show account keys or settings.

Step 2

Scan the chickpea snack label

This sample lists chickpeas and other seasonings. Its nutrition panel says 61 grams of total carbohydrate and 0.5 grams of sugars per 100 grams. The sugar number is low, but the total carbohydrate number is much higher, so sugar alone is not enough to answer the diabetes question.

SHOW / DO: Upload the complete sample label, add a short voice note, then submit. If the live scan is slow, show your prepared result and explain that you are demonstrating the same flow.

Step 3

Explain the caution result

The visible ingredient list does not show peanut, but this image does not establish that the full package's allergen advisory is clear. The app should not turn 'I cannot see a warning here' into 'there is no warning.' It also points out the carbohydrate value instead of judging the product by sugar alone. That is why this is a caution case, not a safe-to-eat guarantee.

SHOW / DO: Point to the missing allergen panel and the 61 g carbohydrates value. The expected current result for this profile is CAUTION.

DEMO BACKUP

Keep the same label image and a screenshot of the expected caution result ready. If the microphone permission, network, or model call fails, say so and use the backup. Never make up a live result.

How the app works

Explain the sequence from the customer's tap to the result. The browser does not call the model directly.

Label photo Voice note Saved needs	Next.js + React Firebase sign-in ID token	Verify token Load profile Handle upload	Whisper tiny Gemini Flash-Lite Python rules	Verdict + reasons Per-user history Shown in UI

What the API does

- The frontend gets a Firebase ID token and sends it as a Bearer token with protected API requests.
- POST /api/analyze accepts a multipart image and audio upload. The FastAPI server checks file-size limits, uses a temporary folder, retrieves the signed-in user's profile, and calls the analysis pipeline.
- GET /api/profile and PUT /api/profile read and save needs. GET /api/history retrieves the user's recent scans. GET /api/health reports that the API is up.
- The response is structured JSON: verdict, summary, ingredients detected, flagged items, restrictions, confidence, and notes. The dashboard renders it and refreshes history.

A SAY-IT-OUT-LOUD SUMMARY

Firebase proves who is signed in. FastAPI checks that proof and coordinates the request. The model services interpret the image and voice. Python safety rules can make the model's proposed verdict more cautious before the result is returned.

What each model does

Whisper tiny (local)	Transcribes the user's voice note to text. Audio is loaded as 16 kHz mono for transcription.	It does not inspect the package or make the food verdict.
Gemini 3.5 Flash-Lite (Google API)	Receives the label image plus the transcribed request and saved profile. It reads the image and returns a JSON analysis.	It is not Gemma 4; its output is not treated as infallible.
Python rules (not a model)	Parses the JSON, normalizes the verdict, and applies safety overrides for uncertainty and selected profile conditions.	The current carbohydrate trigger is a prototype heuristic, not medical guidance.

Safety behavior to describe

- A parse error, invalid verdict, or low-confidence SAFE answer is moved to CAUTION. A SAFE answer with flagged items is upgraded to AVOID.
- For a stated allergy, an unseen allergen-statement area or a detected advisory prevents a non-AVOID result from being called SAFE.
- For diabetes, the current rule checks total carbohydrate separately from sugar. Missing carbohydrate data, or at least 20 g per 100 g, triggers CAUTION. This is an app review trigger only, not a clinical threshold.

PRIVACY: ANSWER PRECISELY

The audio file is uploaded to the app server and transcribed there; temporary upload files are removed after the request. The label image, transcript, and saved profile are sent to Gemini for the analysis. The current local prototype stores profile/history as per-user JSON files and logs some profile/transcript/model text. Do not describe this as production-grade privacy. Remove sensitive logs and move to secure managed storage before release.

Trust, proof, and the close

6:15-7:30

State the limits without weakening the idea

LabelWise is a reading aid, not a medical decision-maker. A SAFE result only means the visible information did not reveal a conflict under the current rules; it cannot guarantee that a package has no cross-contact risk or that an unread part of the label is safe. The confidence field comes from the model and is not a calibrated probability. The diabetes threshold in this prototype is an engineering review trigger, not a clinical recommendation. For severe allergies, the complete package and manufacturer remain the authority.

SHOW / DO: Show the caution language and, if asked, point out the product limitations in the interface.

Evaluation plan - report only numbers you actually collect

- Build a small, labeled photo set with complete, partial, blurry, and multilingual panels, plus clear Contains and May contain statements.
- Measure ingredient/allergen extraction precision and recall, and especially the false-safe rate on allergy cases.
- Time audio transcription, Gemini request, and total scan separately; report median and 95th-percentile latency over repeated runs.
- Compare each model/configuration on the same examples. Keep failed examples and say what changed after fixing them.

7:30-9:00

Roadmap

My next priorities are to show the exact label evidence behind each flag, make image-quality and missing-panel checks more explicit, and evaluate the false-safe cases on a labeled set. I also want to test regional-language voice input and make the profile and data controls clearer. Before calling this a customer-ready service, I would replace the local JSON store, reduce sensitive logging, and validate the health-related rules with qualified reviewers.

SHOW / DO: Show the next-step list, not a promise of features that are not implemented yet.

9:00-10:00

Close with the purpose

The value of LabelWise is not that an AI can say 'safe.' It is that the app can connect a label to a person's stated needs, explain the evidence, and make uncertainty visible. Food labels are personal decisions made in seconds. LabelWise aims to make those seconds clearer and more informed. Thank you - I would love your feedback on how to evaluate it safely for real users.

SHOW / DO: End, then invite questions.

IF THE 10 MINUTES INCLUDE Q&A;

Use 0:00-1:00 opening, 1:00-2:00 solution, 2:00-4:00 demo, 4:00-6:00 architecture, 6:00-7:00 safety, 7:00-8:00 evidence/limits, 8:00-8:30 roadmap, 8:30-10:00 questions. Skip detailed model settings; keep the honest model disclosure.

Judge Q&A;: product and safety

Keep answers direct. If you have not measured or implemented something, say that plainly and give the next step.

1. What is the USP?

It combines a label photo, a spoken question, and a saved personal profile, then returns a reasoned verdict with uncertainty. The focus is helping a person interpret the visible label against their own stated needs, not giving a generic nutrition score.

2. How do you reduce false-safe answers?

The prompt asks for structured evidence and a cautious result. Python overrides malformed output, low-confidence SAFE answers, flagged SAFE answers, missing allergen-statement evidence for an allergy, and missing or high carbohydrate values for the current diabetes rule. These are safeguards, not a guarantee. I still need a larger labeled evaluation set.

3. Does SAFE guarantee the food is safe?

No. It only means the visible label and current checks did not identify a conflict. It cannot verify manufacturing cross-contact, a missing panel, or information that is not on the package. People with severe allergies should verify the full package and contact the manufacturer when uncertain.

4. How does it handle diabetes?

It extracts total carbohydrates and their basis separately from sugars. The current prototype returns CAUTION if the carb value is absent or reaches its 20 g per 100 g review trigger. That number is an engineering heuristic, not a medical threshold; it needs expert review and serving-size-aware evaluation.

5. Can the model hallucinate ingredients?

Yes. That is a known limitation of image understanding. The app should show the detected ingredients and notes, ask for a clearer/full label when needed, default to caution when uncertain, and be measured against manually labeled examples. It should not hide the source text from the user.

6. Is the confidence score calibrated?

No. It is the model's categorical confidence, not a statistically calibrated probability that the verdict is correct. Calibration and selective abstention are part of the evaluation work I would do next.

7. What if the photo is cropped or blurry?

The system is instructed to mark partial or unreadable label information as uncertain, and the local allergy rule will not call it SAFE when the allergen statement area is not confirmed. A stronger next version should detect panel coverage and request another photo before analysis.

8. Why use AI instead of only OCR and rules?

The current Gemini model reads the package image and interprets the user's request in context. Rules then enforce a few conservative checks. A dedicated OCR and evidence-highlighting stage is a valuable next addition because it would make the output more inspectable and easier to test.

Judge Q&A;: models, data, and readiness

9. Which models are running in the current demo?

Gemini 3.5 Flash-Lite handles the image-plus-text label analysis through the Google API. Whisper tiny runs locally on the backend to transcribe the voice note. Python code parses the model's JSON and applies deterministic verdict overrides. The current live code does not use Gemma 4.

10. How does this meet the Gemma 4 track?

Use this exact disclosure if Gemma 4 is not integrated by presentation time: 'The live prototype currently uses Gemini Flash-Lite for image analysis and Whisper tiny for transcription. I have not yet integrated Gemma 4 into this live path, so I will not claim that it powers this demo. My next step is to put Gemma 4 in the core analysis path and benchmark it on the same safety examples.' If the judging track requires a working Gemma 4 integration, this gap may affect eligibility or scoring.

11. How does sign-in and the API work?

Firebase handles email/password and Google sign-in. The browser gets an ID token and sends it as a Bearer token. FastAPI verifies it with Firebase Admin, uses the token's UID to load and save that user's profile/history, then returns the analysis JSON. The browser does not send the service-account key to the frontend.

12. What data is stored or sent outside the app?

The prototype stores profiles and scan summaries in per-user JSON files on the backend. The temporary image and audio files are removed after the request, but the label image, transcript, and saved profile are sent to Gemini for analysis. The current code also logs some transcript/profile/model text; before external use those logs need minimization and a clear retention policy.

13. Is it under 15 seconds?

I do not have a benchmark that proves that yet. The backend warms Whisper at startup, but scan time still depends on the voice note, hardware, network, and Gemini response. The code logs transcription, model-call, and total times. I will report median and 95th-percentile results after repeated end-to-end runs rather than promise a number now.

14. Is the data store production-ready?

No. It is a simple local JSON-per-user store for this prototype. A real hosted service needs managed storage, access controls, deletion/export controls, retention rules, safer logs, monitoring, and rate limits.

15. What is the next milestone?

Build a manually labeled test set, measure false-safe cases and latency, add visible evidence for each conclusion, and place Gemma 4 in a real core role if the track requires it. I would only describe features and results that are present and measured.

LAST-MINUTE CHECKLIST

[ ] Run API: **python app.py**
[ ] Run web app: **cd web**, then **npm run dev**
[ ] Confirm sign-in and one scan before the event
[ ] Keep the chickpea sample and a backup result ready
[ ] Do not show API keys, service-account JSON, or raw logs
[ ] Say 'prototype' and separate measured results from goals

Reference notes: Current implementation details are based on the LabelWise repository. For India-specific label rules, check the latest FSSAI Labelling and Display amendments before describing regulatory coverage. Gemma 4 official overview: multimodal image, text, and audio capabilities vary by model variant.
FSSAI labelling amendments | Gemma 4 overview